

REGRAS LÓGICAS E SIMPLIFICAÇÃO BOOLEANA

Expressão Booleana Original

Nas especificações iniciais do sistema de monitoramento da colônia, o disparo e a classificação de um Alerta Crítico exigiam a verificação simultânea de múltiplos estados operacionais. A regra de negócio original era representada pela seguinte expressão booleana:

$$\text{ALERTA_CRITICO} = (\text{FALHA} \wedge \text{CRITICO}) \vee (\text{FALHA} \wedge \neg \text{CRITICO})$$

Onde as variáveis lógicas assumem os valores:

- FALHA: Indica a ocorrência de uma anomalia ou falha estrutural/operacional no módulo (True ou False).
- CRITICO: Indica o nível de severidade ou criticidade adicional atribuído ao evento (True ou False).

Desenvolvimento da Simplificação Algébrica

Para eliminar redundâncias lógicas e reduzir o custo computacional das validações executadas pelo script em Python, aplicamos o Teorema da Simplificação da álgebra booleana ($X \cdot Y + X \cdot \sim Y = X$):

1. **Fatoração por Distributiva (Evidenciando o termo comum FALHA):**

$$\text{ALERTA_CRITICO} = \text{FALHA AND } (\text{CRITICO OR NOT CRITICO})$$

2. **Aplicação da Lei do Complemento (CRITICO OR NOT CRITICO):**

$$\text{ALERTA_CRITICO} = \text{FALHA AND } 1$$

3. **Aplicação da Lei da Identidade (FALHA AND 1 = FALHA):**

$$\text{ALERTA_CRITICO} = \text{FALHA}$$

Justificativa e Implementação no Código

A simplificação matemática demonstra que a variável de criticidade secundária CRITICO é totalmente redundante para o disparo primário do alerta. Se houver uma falha catalogada, o evento já é considerado crítico por definição.

No código-fonte do projeto, essa lógica otimizada foi implementada na função `validar_alerta_critico`, garantindo um código limpo e direto:

```

209 def validar_alerta_critico(alerta: dict) -> bool:
210     """
211     Regra de negócio original (item 1.4 do enunciado):
212     |   ALERTA_CRITICO = (FALHA AND CRITICO) OR (FALHA AND NOT CRITICO)
213
214     Aplicando o Teorema da Simplificação ( $X.Y + X.\sim Y = X$ ), com  $X = FALHA$ 
215     e  $Y = CRITICO$ , a variável CRITICO se cancela e a regra se reduz a:
216     |   ALERTA_CRITICO = FALHA
217
218     Ou seja: para o disparo do alerta importa apenas se houve falha –
219     a criticidade adicional não muda o resultado da expressão.
220     """
221
222     if not isinstance(alerta, dict):
223         return False
224     return alerta.get("tipo_ocorrencia") == "falha_critica"
225

```

Controle de Acesso e Aplicação das Leis de De Morgan

Além da triagem de alertas, o NCAS implementa uma segunda regra lógica focada na segurança do sistema, a liberação de consultas a módulos restritos.

- Condição de Liberação Original: O acesso à base de dados do módulo é permitido somente se o operador estiver autenticado e o módulo estiver em estado operacional.

$$\text{LIBERAR} = \text{AUTORIZADO} \wedge \text{ATIVO}$$

- Condição de Bloqueio (Aplicação das Leis de De Morgan): Para otimizar a checagem de exceções (quando o acesso deve ser negado), aplicamos a negação da expressão de liberação utilizando as Leis de De Morgan ($\neg(A \wedge B) = \neg A \vee \neg B$):

$$\begin{aligned} \text{BLOQUEAR} &= \neg(\text{AUTORIZADO} \wedge \text{ATIVO}) \\ \text{BLOQUEAR} &= \neg\text{AUTORIZADO} \vee \neg\text{ATIVO} \end{aligned}$$

A aplicação de De Morgan simplifica a tomada de decisão computacional na Opção 8 do sistema (*verificar_liberacao_consulta*), atestando que a consulta será bloqueada instantaneamente caso o usuário não esteja autorizado ou o módulo esteja inativo, evitando condicionais aninhadas complexas e tornando o fluxo do software mais legível e eficiente.